Roteiro de Aula – Introdução às Árvores de Busca Binária (BST)

1. Definição e Fundamentação Teórica

Contextualização

O ensino de Estruturas de Dados avançou para enfatizar o raciocínio sobre invariantes de estruturas de dados e o equilíbrio de *trade-offs* entre tempo e espaço. A transição de coleções lineares (vetores e matrizes) para estruturas hierárquicas dinâmicas (Árvores) é um marco fundamental nessa formação.

- O Dilema das Estruturas Lineares:
  - Em vetores não ordenados, a inserção é ultra-rápida (O(1)), mas a busca exige a verificação item a item com `equals`, resultando em tempo linear O(n).
  - Em vetores ordenados, a Busca Binária atinge altíssima eficiência de busca (O($\log_2$ n)), contudo inserções e remoções são ineficientes (O(n)), pois exigem o deslocamento contínuo de milhares de elementos na memória contígua.
- **A Solução da Árvore de Busca Binária (BST)**:

A **Árvore de Busca Binária** (*Binary Search Tree - BST*) resolve este dilema combinando a eficiência de busca logarítmica (O($\log_2$ n)) com a flexibilidade da alocação dinâmica de memória por ponteiros/referências. Na BST, inserções e remoções ocorrem de forma local sem a necessidade de reordenar ou deslocar um bloco contínuo de elementos.

Conceitos Fundamentais e Terminologia

Uma **Árvore** é um conjunto finito de nós (*nodes*) conectados por arestas de forma hierárquica (relação pai-filho):

- **Raiz (*Root*)**: O nó principal no topo da hierarquia (nível 0) que não possui pai.
- **Nó Interno**: Qualquer nó que possui pelo menos um nó filho.
- **Folha ou Nó Externo (*Leaf*)**: Nó de grau 0, ou seja, que não possui nenhum filho.
- **Grau de um Nó**: Número de subárvores/filhos que emergem do nó. Em uma Árvore Binária, o grau máximo de qualquer nó é **2**.
- **Subárvore**: Estrutura recursiva na qual cada nó atua como a raiz de sua própria subárvore (esquerda ou direita).
- **Nível e Profundidade**: O nível da raiz é 0. Os filhos diretos da raiz estão no nível 1, seus netos no nível 2, e assim por diante.
- **Altura de um Nó (H)**: O número de nós (ou arestas) do maior caminho entre o nó e uma folha descendente. A altura de uma árvore com um único nó é **0**; a altura de uma árvore vazia é **-1**.

Propriedades Matemáticas Essenciais

1. **Número máximo de nós no nível n**: 2^n

2. **Número máximo de nós em uma árvore de altura H**: $2^{H+1} - 1$
3. **Invariante da Árvore de Busca Binária (BST Invariant)**:

Para todo nó N existente na árvore:

- Todos os nós da subárvore **esquerda** contêm valores estritamente **menores** que N (sub_esquerda < N).
- Todos os nós da subárvore **direita** contêm valores estritamente **maiores** (ou maiores/iguais) que N (sub_direita ≥ N).

Aplicações no Mundo Real e Bancos de Dados

- **Índices de Bancos de Dados (B-Trees / B+Trees)**: Gerenciadores de banco de dados relacionais (PostgreSQL, MySQL, Oracle, SQL Server) utilizam árvores balanceadas de busca para implementar índices em tabelas com milhões de registros. Diferente de tabelas Hash (que funcionam apenas para busca exata `WHERE id = 10`), árvores mantêm os dados ordenados, permitindo consultas por intervalo (`WHERE data BETWEEN '2026-01-01' AND '2026-12-31'`) e ordenação automática (`ORDER BY`).
- **Sistemas de Arquivos**: Organização de diretórios e arquivos em sistemas operacionais.
- **Árvores de Expressão Aritmética e Compiladores**: Construção de Árvores de Sintaxe Abstrata (AST) para interpretar e otimizar código-fonte.

---

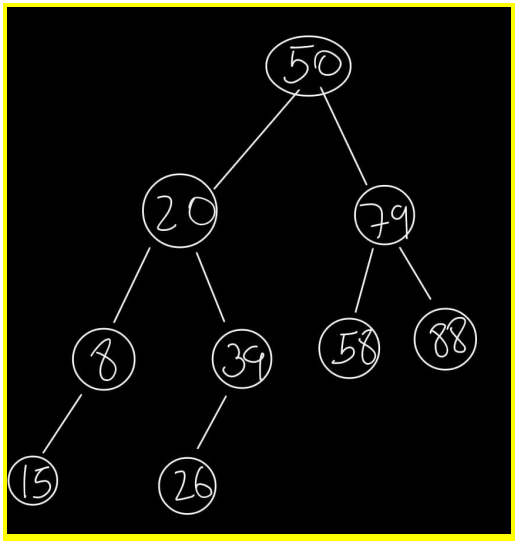
2. Simule a execução

Construção de uma Árvore de Busca Binária (BST)

Considere a sequência de chaves a serem inseridas em uma BST inicialmente vazia:

V = [50, 20, 39, 8, 79, 26, 58, 15, 88]

1. Simule o processo de inserção nó a nó aplicando a da BST (menor → esquerda, maior → direita)



Pergunta 1: Qual é a altura (H) da árvore resultante da Simulação 1 e quais nós são considerados **folhas** (*leaves*)?

3, FOLHAS 15, 26, 58, 88

Pergunta 2: Se os mesmos elementos fossem inseridos na ordem perfeitamente ordenada V_{ord} = [8, 15, 20, 26, 39, 50, 58, 79, 88], qual seria o formato da árvore resultante e qual a sua altura?

FICA LINHA RETA, BASICAMENTE UMA LISTA LINEAR, ALTURA 8

3. Código da classe `Node`

Abaixo está a implementação genérica em Java utilizando a interface `Comparable<T>` para garantir que qualquer tipo ordenável possa ser armazenado na árvore.

```
// Classe representando um Nó genérico da Árvore Binária
public class Node<T extends Comparable<T>> {
    T dado;
    Node<T> esquerda;
    Node<T> direita;

    public Node(T dado) {
        this.dado = dado;
        this.esquerda = null;
        this.direita = null;
    }
}
```

Fique ligado: premissas e conclusões da aula!

- A **Árvore de Busca Binária (BST)** atinge busca, inserção e remoção em tempo logarítmico **O(log₂ n)** no caso médio, sem necessitar da realocação ou deslocamento contínuo de memória de vetores.
- O percurso recupera automaticamente todos os dados em ordem estritamente crescente sem nenhum custo adicional de algoritmo de ordenação.
- **Atenção ao Pior Caso (Árvore Degenerada)**: Se os dados forem inseridos na árvore já ordenados (ex: 1, 2, 3, 4, 5), a árvore se desbalanceia totalmente e se transforma em uma Lista Encadeada Simples, degradando a busca para O(n). Para prevenir essa falha em sistemas de produção, utilizam-se árvores auto-balanceadas (AVL, Red-Black Trees e B-Trees).

Questões Finais

Pergunta 3: Em bancos de dados relacionais com milhões de registros, por que índices em árvore (B-Tree/B+Tree) são preferidos a tabelas Hash quando a consulta SQL contém `WHERE valor BETWEEN 100 AND 500` ou `ORDER BY data`?

pq pra hash eu consigo fazer comparação de igualdade exata, só que arvore permite fazer consultas baseadas em intervalos.

Pergunta 4: Qual é o impacto no uso de memória ao substituir um vetor primitivo `int[]` por uma Árvore Binária em Java?a árvore binária consome mais memória que um vetor primitivo, pois além do dado precisa armazenar os ponteiros para os filho